Embedding Python Code in UltiSnips Snippets

Using Python interpolation to generate dynamic content inside Vim snippet expansions

Ronald 'Ryy' G. Thomas

2025-07-20

Table of contents

1 Introduction **2**
- 1.1 Motivations . . . 2
- 1.2 Objectives . . . 2

2 Prerequisites and Setup **3**

3 What is UltiSnips Python Interpolation? **3**

4 Getting Started: Basic Python Snippets **3**
- 4.1 The Chord Generator: Solving the Stack Overflow . . . 4
- 4.2 Advanced Patterns . . . 5

5 Debugging Python Snippets **5**
- 5.1 Things to Watch Out For . . . 6
- 5.2 Lessons Learnt . . . 6
  - 5.2.1 Conceptual Understanding . . . 6
  - 5.2.2 Technical Skills . . . 7
  - 5.2.3 Gotchas and Pitfalls . . . 7
- 5.3 Limitations . . . 7
- 5.4 Opportunities for Improvement . . . 7

6 Wrapping Up **7**

7 See Also **8**

8 Reproducibility **8**

Figure 1: Python and Vim integration through UltiSnips enables dynamic, programmable snippet expansions.

When static snippets are not enough, Python interpolation fills the gap.

1 Introduction

[YOUR CONTENT HERE – 2-3 paragraphs. Open with an "I didn't really know…" hook about UltiSnips Python interpolation. Reference the Stack Overflow question that prompted this post: https://stackoverflow.com/questions/78636197

Describe the problem: needing a snippet that generates a variable number of elements (e.g., music chord placeholders). Transition to what this post will cover.]

1.1 Motivations

[YOUR CONTENT HERE – 4-6 bullet points. Personal and specific reasons. Examples: static snippets were too rigid, needed parametric expansion, wanted to automate repetitive patterns in code, saw the Stack Overflow question and wanted to solve it, etc.]

1.2 Objectives

1. [YOUR CONTENT HERE – specific, measurable]
2. [YOUR CONTENT HERE]

3. [YOUR CONTENT HERE]
4. [YOUR CONTENT HERE]

I am documenting my learning process here. If you spot errors or have better approaches, please let me know.

Vim's extensibility through UltiSnips and Python creates a powerful text generation system.

2 Prerequisites and Setup

[YOUR CONTENT HERE – 1-2 paragraphs. What readers need: Vim 8+ compiled with Python3 support, UltiSnips plugin installed (via vim-plug, Vundle, or pathogen). Show how to verify Python support.]

```
" Check Python support in Vim
:echo has('python3')

" UltiSnips configuration in .vimrc
Plug 'SirVer/ultisnips'
let g:UltiSnipsExpandTrigger = "<tab>"
let g:UltiSnipsJumpForwardTrigger = "<c-j>"
let g:UltiSnipsJumpBackwardTrigger = "<c-k>"
```

3 What is UltiSnips Python Interpolation?

[YOUR CONTENT HERE – 1-2 paragraphs in plain language. Explain that UltiSnips supports three interpolation types: shell (`), VimScript (!v), and Python (!p). Python interpolation executes Python code at snippet expansion time and inserts the result. Use an analogy – e.g., a regular snippet is a rubber stamp, but Python interpolation is a mail merge that customises each stamp.]

4 Getting Started: Basic Python Snippets

[YOUR CONTENT HERE – First code examples. Start with simple Python interpolation: current date, simple string manipulation. Follow CODE -> OUTPUT -> INTERPRETATION. Show the snippet definition and what it expands to.]

```
# [YOUR SNIPPET HERE -- basic Python example]
snippet today "Insert today's date"
`!p snip.rv = __import__('datetime').date.today().isoformat()`
endsnippet
```

[YOUR INTERPRETATION HERE – 2-4 sentences explaining the syntax: `!p` marker, snip.rv, Python imports.]

```
# [YOUR SNIPPET HERE -- second basic example]
```

[YOUR INTERPRETATION HERE]

Figure 2: A research library environment representing the study phase of learning a new programming technique.

Learning the interpolation syntax opens up a wide range of dynamic snippet possibilities.

4.1 The Chord Generator: Solving the Stack Overflow

Problem

[YOUR CONTENT HERE – The core section. Show the original Stack Overflow problem: generating a variable number of chord placeholders. Present the Python function from the original post:]

```
def make_chords(num_reps):
    text = ""
    for i in range(num_reps):
        text += "< > "
    return text
```

[YOUR CONTENT HERE – Show how to embed this in an UltiSnips snippet definition. Walk through the syntax step by step.]

```
# [YOUR SNIPPET HERE -- chord generator]
```

[YOUR INTERPRETATION HERE]

4.2 Advanced Patterns

[YOUR CONTENT HERE – Show 2-3 more advanced Python interpolation patterns. Examples: generating numbered lists, creating boilerplate code with parameters, conditional content based on filetype. Each with CODE -> OUTPUT -> INTERPRETATION.]

```
# [YOUR SNIPPET HERE -- advanced example 1]
```

[YOUR INTERPRETATION HERE]

```
# [YOUR SNIPPET HERE -- advanced example 2]
```

[YOUR INTERPRETATION HERE]

5 Debugging Python Snippets

[YOUR CONTENT HERE – How to debug when Python interpolation fails. Show common error messages, how to check Vim's messages, how to test Python code outside of UltiSnips.]

```
" [YOUR DEBUG COMMANDS HERE]
```

[YOUR INTERPRETATION HERE]

5.1 Things to Watch Out For

1. **[YOUR GOTCHA HERE]** – [explanation]
2. **[YOUR GOTCHA HERE]** – [explanation]
3. **[YOUR GOTCHA HERE]** – [explanation]
4. **[YOUR GOTCHA HERE]** – [explanation]
5. **[YOUR GOTCHA HERE]** – [explanation]

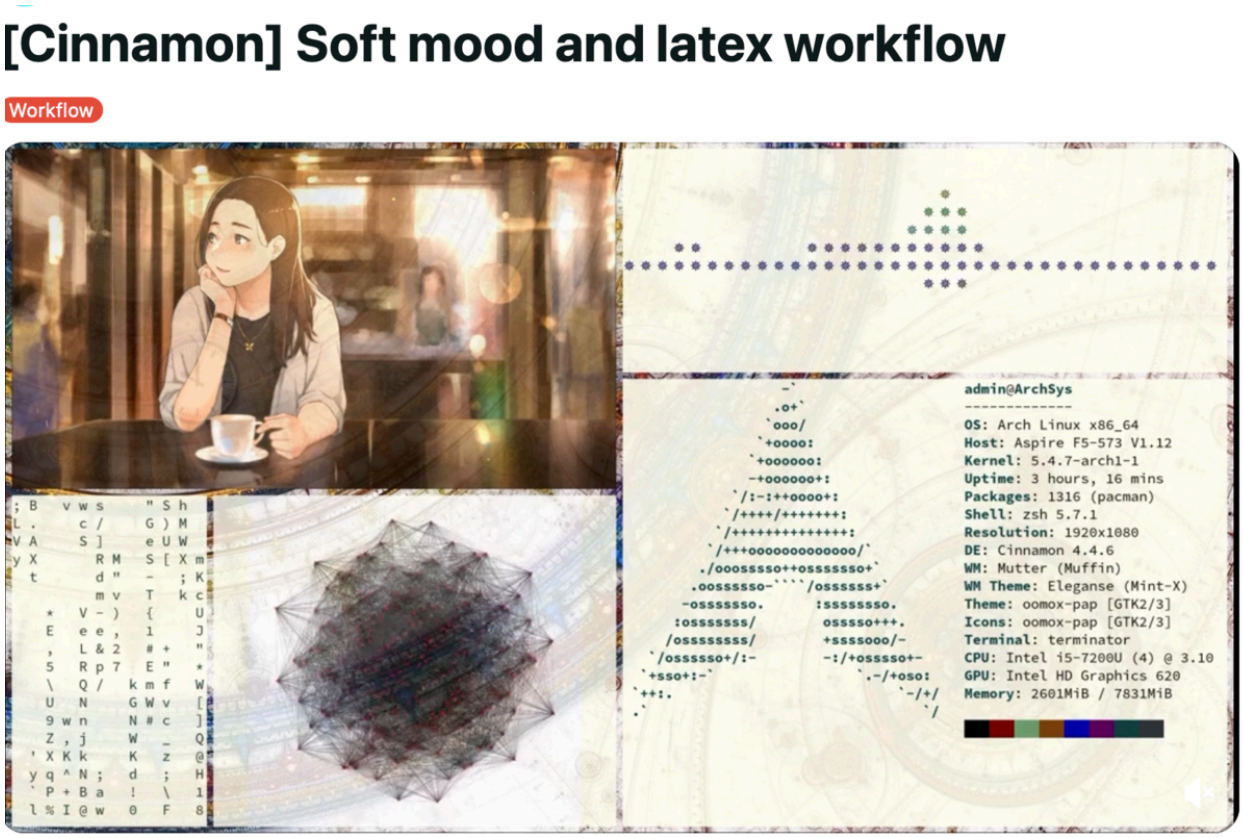

Figure 3: A calm workspace at the end of a productive coding session, representing the completion of a Vim customisation project.

A well-crafted snippet library compounds productivity gains over months and years.

5.2 Lessons Learnt

5.2.1 Conceptual Understanding

- [YOUR CONTENT HERE – what you learnt about UltiSnips architecture and Python interpolation]
- [YOUR CONTENT HERE]

- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]

5.2.2 Technical Skills

- [YOUR CONTENT HERE – tools/techniques mastered: snip.rv, tabstops with Python, importing modules, etc.]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]

5.2.3 Gotchas and Pitfalls

- [YOUR CONTENT HERE – practical warnings for future UltiSnips users]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]

5.3 Limitations

- [YOUR CONTENT HERE – 5-6 bullets. Cover: requires Python3-compiled Vim, UltiSnips not available in Neovim natively (use LuaSnip), Python code runs synchronously, security concerns with untrusted snippets, etc.]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]

5.4 Opportunities for Improvement

1. [YOUR CONTENT HERE – extensions: LuaSnip equivalent for Neovim, snippet library sharing, more complex generators, etc.]
2. [YOUR CONTENT HERE]
3. [YOUR CONTENT HERE]
4. [YOUR CONTENT HERE]
5. [YOUR CONTENT HERE]
6. [YOUR CONTENT HERE]

6 Wrapping Up

[YOUR CONTENT HERE – 2-3 paragraphs. First: recap the main technique. Second: what you learnt personally. Third: advice for readers.]

Main takeaways:

- [YOUR CONTENT HERE – 3-4 specific results]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]
- [YOUR CONTENT HERE]

7 See Also

Related posts:

- "Writing a Simple Vim Plugin for REPL Interaction" (complementary Vim customisation)
- [YOUR RELATED POST 2]

Key resources:

- UltiSnips Documentation – Official UltiSnips reference
- UltiSnips Python Interpolation (Vimcasts) – Video tutorial on Python snippets
- Stack Overflow: Python in UltiSnips – The question that prompted this post
- LuaSnip (Neovim alternative) – Lua-based snippet engine for Neovim
- Vim Python Interface Documentation – Official Vim-Python integration reference

8 Reproducibility

[YOUR CONTENT HERE – environment description: Vim version, Python version, UltiSnips version.]

File	Purpose
[YOUR FILES HERE]	[descriptions]

```
" To verify your setup:
:echo has('python3')
:echo g:UltiSnipsExpandTrigger
```

9 Let's Connect

- **GitHub:** rgt47
- **Twitter/X:** @rgt47
- **LinkedIn:** Ronald Glenn Thomas
- **Email:** rgtlab.org/contact

I would enjoy hearing from you if:

- You spot an error or a better approach to any of the code in this post.
- You have suggestions for topics you would like to see covered.

- You want to discuss R programming, data science, or reproducible research.
- You have questions about anything in this tutorial.
- You just want to say hello and connect.